

Corso Python

Modulo 7: Python Backend & APIs



Collegamento al Modulo Precedente

Nel **Modulo 6** abbiamo imparato:

- NumPy per calcoli vettoriali
- Pandas avanzato (pulizia, groupby, merge)
- Pipeline di automazione dati
- Esportazione report (Excel, CSV)

Ora esponiamo i dati al mondo:

- **API REST:** Architettura e protocollo HTTP
- **FastAPI:** Framework moderno per creare API
- **Pydantic:** Validazione dati robusta

Agenda del Modulo (4 Ore)

Esporre i nostri script al mondo tramite il Web.

1. **Cos'è un'API?** Concetti fondamentali e architettura REST.
2. **HTTP:** Metodi, status code e anatomia delle richieste.
3. **FastAPI:** Il framework moderno per le API Python.
4. **Routing:** Path parameters, query parameters, request body.
5. **Pydantic:** Validazione dati robusta e automatica.
6. **CRUD Completo:** Create, Read, Update, Delete.
7. **Laboratorio:** Laboratorio riassuntivo del corso.

1. Cos'è un'API?

Application Programming Interface

API: Il Cameriere del Software

Un'**API** (Application Programming Interface) è un'interfaccia che permette a due software di comunicare tra loro.

Analogia del ristorante:

- Tu (Client) → vuoi ordinare del cibo
- La Cucina (Server) → prepara il cibo, ma non puoi entrarci
- Il Cameriere (API) → prende l'ordine e porta il piatto



API

Perché le API sono fondamentali?

- **Astrazione:** Non devi sapere COME funziona internamente, solo COSA chiedere
- **Contratto:** L'API definisce regole precise (formato, parametri, risposte)
- **Sicurezza:** Il client non accede mai direttamente ai dati/logica interna

Perché le API sono Ovunque?

Quando usi...	Stai chiamando un'API di...
Google Maps nel tuo sito	Google Maps API
Login con Google/Facebook	OAuth API
Pagamenti online	Stripe/PayPal API
ChatGPT	OpenAI API

Le API permettono di:

- Riutilizzare funzionalità esistenti
- Separare frontend e backend
- Integrare servizi di terze parti
- Creare ecosistemi di applicazioni

REST: Lo Standard delle API Web

REST (Representational State Transfer) è un'architettura per progettare API web, definita da Roy Fielding nel 2000.

Principi chiave di REST:

Principio	Significato	Beneficio
Stateless	Ogni richiesta è indipendente	Scalabilità, semplicità
Client-Server	Separazione responsabilità	Evolgono indipendentemente
Uniform Interface	URL coerenti e prevedibili	Facile da imparare e usare
Resource-based	Tutto è una "risorsa"	Design intuitivo

API REST

Cosa significa "Stateless"?

Il server non ricorda le richieste precedenti. Ogni richiesta deve contenere TUTTE le informazioni necessarie (es. token di autenticazione).

Esempio di URL RESTful:

GET	/users	→ Lista utenti
GET	/users/42	→ Dettaglio utente 42
POST	/users	→ Crea nuovo utente
PUT	/users/42	→ Modifica utente 42
DELETE	/users/42	→ Elimina utente 42

Nota: L'URL identifica la **risorsa**, il metodo HTTP indica l'**azione**.

2. HTTP: Il Protocollo del Web

Metodi, Status Code e Struttura

I Metodi HTTP (Verbi)

Ogni richiesta HTTP ha un **metodo** che indica l'azione da compiere sulla risorsa.

Metodo	Azione	Idempotente?	Body?	Uso tipico
GET	Leggere dati	✓	No	Recuperare risorse
POST	Creare risorsa	✗	Sì	Form, nuovi record
PUT	Sostituire risorsa	✓	Sì	Update completo
PATCH	Modifica parziale	✗	Sì	Update singolo campo
DELETE	Eliminare risorsa	✓	No	Rimozione

Metodi Idempotenti

Cosa significa "Idempotente"?

Chiamarlo **N volte** ha lo stesso effetto di chiamarlo **1 volta**.

- `DELETE /users/42` → La prima volta elimina, le successive non cambiano nulla
- `POST /users` → Ogni chiamata crea un NUOVO utente (non idempotente!)

Perché è importante?

Se una richiesta fallisce (timeout), puoi riprovare un'operazione idempotente senza rischi. Con POST, potresti creare duplicati!

Status Code HTTP

Il server risponde con un **codice numerico** che indica l'esito della richiesta. Conoscerli è fondamentale per debugging e design.

Range	Categoria	Significato
2xx	✓ Successo	La richiesta è andata a buon fine
3xx	↺ Redirect	La risorsa si è spostata
4xx	✗ Errore Client	Hai sbagliato qualcosa TU
5xx	💥 Errore Server	Ha sbagliato qualcosa il SERVER

Status Code HTTP

I più importanti da ricordare:

Codice	Nome	Quando usarlo
200	OK	Richiesta GET/PUT riuscita
201	Created	Risorsa creata con POST
204	No Content	DELETE riuscito (niente da restituire)
400	Bad Request	JSON malformato, parametri mancanti
401	Unauthorized	Token mancante o invalido
403	Forbidden	Autenticato ma non autorizzato
404	Not Found	Risorsa non esiste
422	Unprocessable	Validazione fallita (FastAPI)
500	Internal Error	Bug nel server

Anatomia di una Richiesta HTTP

```
POST /api/users HTTP/1.1      ← Metodo + Path + Versione
Host: api.example.com         ← Headers
Content-Type: application/json
Authorization: Bearer token123

{                               ← Body (JSON)
  "name": "Mario Rossi",
  "email": "mario@example.com"
}
```

Componenti:

- **URL/Path:** Dove mandare la richiesta
- **Headers:** Metadati (autenticazione, formato, ecc.)
- **Body:** I dati da inviare (solo POST/PUT/PATCH)

JSON: Il Formato dei Dati

Le API moderne usano **JSON** (JavaScript Object Notation) per scambiare dati.

```
{
  "id": 1,
  "name": "Mario Rossi",
  "email": "mario@example.com",
  "active": true,
  "roles": ["admin", "user"],
  "profile": {
    "age": 30,
    "city": "Roma"
  }
}
```


JSON vs XML

Perché JSON ha vinto su XML?

Aspetto	JSON	XML
Verbosità	Compatto	Molto verbose
Parsing	Nativo in JS, facile ovunque	Più complesso
Leggibilità	Alta	Media
Tipi dati	string, number, bool, null, array, object	Solo stringhe

JSON ↔ **Python**: La conversione è naturale!

3. FastAPI

Il Framework Moderno per API Python

Perché FastAPI?

Esistono molti framework Python per API (Flask, Django, Bottle...), ma **FastAPI** è diventato lo standard moderno dal 2018.

Caratteristica	Flask	Django REST	FastAPI
Performance	Media	Media	Alta (async)
Validazione auto	✗ Manuale	Parziale	✓ Pydantic
Docs automatiche	✗ Plugin	✗ Plugin	✓ Built-in
Async nativo	✗	Parziale	✓ Completo
Type Hints	Opzionale	Opzionale	Core
Curva apprendimento	Bassa	Alta	Bassa

Caratteristiche FastAPI

Cosa rende FastAPI speciale?

1. **Type Hints = Validazione:** I tipi Python diventano regole di validazione
2. **Docs gratuite:** Swagger UI generata automaticamente dal codice
3. **Async-first:** Supporto nativo per operazioni I/O non bloccanti
4. **Modern Python:** Sfrutta le feature di Python 3.7+

Sotto il cofano: FastAPI usa **Starlette** (web framework async) e **Pydantic** (validazione).

Installazione e Setup

```
# Installa FastAPI e il server ASGI
pip install fastapi "uvicorn[standard]"
```

Componenti:

- `fastapi` → Il framework
- `uvicorn` → Server ASGI per eseguire l'app

Struttura progetto tipica:

```
my_api/
├── main.py           # Entry point
├── models.py         # Modelli Pydantic
├── routers/          # Endpoint organizzati
│   ├── users.py
│   └── items.py
└── requirements.txt
```

La Prima API: Hello World

Bastano **5 righe** di codice per avere un server web funzionante.

```
# main.py
from fastapi import FastAPI

app = FastAPI() # Crea l'applicazione

@app.get("/") # Decoratore: metodo GET, path "/"
def read_root():
    return {"message": "Hello World"}
```

Esecuzione:

```
uvicorn main:app --reload
```

- `--reload` → riavvia automaticamente se modifichi il codice

Testare l'API

Con il server in esecuzione, apri il browser:

URL	Cosa vedi
<code>http://127.0.0.1:8000/</code>	<code>{"message": "Hello World"}</code>

Oppure da terminale con curl:

```
curl http://127.0.0.1:8000/
```



Esercizi Intermedi: FastAPI Base

Pratica Prima di Proseguire

Esercizio 2.1: Primi Endpoint

Obiettivo: Creare un'applicazione FastAPI con endpoint GET di base.

```
from fastapi import FastAPI
app = FastAPI()
# Task:
# 1. Crea un endpoint GET su "/" che restituisce
#    {"app": "Il Mio Primo Server", "versione": "1.0"}
#
# 2. Crea un endpoint GET su "/info" che restituisce
#    {"autore": "Il tuo nome", "linguaggio": "Python"}
#
# 3. Crea un endpoint GET su "/saluto" che restituisce
#    {"messaggio": "Ciao dal server FastAPI!"}
#
# 4. Avvia il server con: uvicorn main:app --reload
#    e testa ogni endpoint nel browser
```

Esercizio 2.2: Endpoint con Dati Statici

Obiettivo: Restituire strutture dati più complesse da endpoint.

```
from fastapi import FastAPI
app = FastAPI()
# Dati statici (simulano un mini-database)
menu = [
    {"id": 1, "piatto": "Margherita", "prezzo": 8.0},
    {"id": 2, "piatto": "Diavola", "prezzo": 10.0},
    {"id": 3, "piatto": "Quattro Formaggi", "prezzo": 11.0},
]

# Task:
# 1. Crea un endpoint GET "/menu" che restituisce la lista completa
# 2. Crea un endpoint GET "/menu/count" che restituisce
#    il numero totale di piatti: {"totale": 3}
# 3. Crea un endpoint GET "/menu/prezzi" che restituisce
#    il prezzo medio: {"prezzo_medio": ...}
```

4. Routing e Parametri

Path, Query e Body

Path Parameters

Valori dinamici **nell'URL** stesso.

```
# URL: http://localhost:8000/users/42
@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}

# URL: http://localhost:8000/files/documents/report.pdf
@app.get("/files/{file_path:path}")
def get_file(file_path: str):
    return {"file_path": file_path}
```

Validazione automatica:

FastAPI usa i Type Hints (`: int`) per validare. Se passi una lettera dove serve un numero
→ errore 422 automatico!

Query Parameters

Valori passati **dopo il** `?` nell'URL.

```
# URL: http://localhost:8000/items?skip=10&limit=5
@app.get("/items")
def list_items(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}

# Parametri opzionali
from typing import Optional

@app.get("/search")
def search(q: Optional[str] = None, category: str = "all"):
    if q:
        return {"query": q, "category": category}
    return {"category": category}
```

Path vs Query

Differenza Path vs Query - Quando usare quale?

Tipo	Sintassi URL	Uso	Esempio
Path	<code>/users/42</code>	Identificare UNA risorsa specifica	ID, slug, username
Query	<code>/items?limit=10</code>	Filtrare, ordinare, paginare	Filtri, ricerca, opzioni

Regola pratica:

- Se è **obbligatorio** per identificare la risorsa → Path
- Se è **opzionale** o modifica il risultato → Query

Combinare Path e Query

```
# URL: http://localhost:8000/users/42/orders?status=pending&limit=5
@app.get("/users/{user_id}/orders")
def get_user_orders(
    user_id: int,                # Path parameter
    status: Optional[str] = None, # Query parameter opzionale
    limit: int = 10              # Query parameter con default
):
    return {
        "user_id": user_id,
        "status": status,
        "limit": limit
    }
```

Regola: I parametri nella firma della funzione che NON sono nel path diventano automaticamente query parameters.



Esercizi Intermedi: Routing e Parametri

Pratica Prima di Proseguire

Esercizio 3.1: Path Parameters

Obiettivo: Creare endpoint con parametri dinamici nel path.

```
from fastapi import FastAPI

app = FastAPI()

studenti = {
    1: {"nome": "Alice", "voto": 28},
    2: {"nome": "Bob", "voto": 25},
    3: {"nome": "Charlie", "voto": 30},
}

# Task:
# 1. Crea GET "/studenti/{studente_id}" che restituisce
#     i dati dello studente (usa studente_id: int)
# 2. Cosa succede se chiami /studenti/99 (ID inesistente)?
#     Aggiungi un controllo che restituisce
#     {"errore": "Studente non trovato"} se l'ID non esiste
# 3. Crea GET "/saluta/{nome}" che restituisce
#     {"messaggio": "Ciao, {nome}!"}
```

Esercizio 3.2: Query Parameters

Obiettivo: Usare query parameters per filtrare e paginare risultati.

```
from fastapi import FastAPI
from typing import Optional

app = FastAPI()

prodotti = [
    {"id": 1, "nome": "Laptop", "prezzo": 999, "categoria": "elettronica"},
    {"id": 2, "nome": "Mouse", "prezzo": 25, "categoria": "accessori"},
    {"id": 3, "nome": "Monitor", "prezzo": 299, "categoria": "elettronica"},
    {"id": 4, "nome": "Tastiera", "prezzo": 75, "categoria": "accessori"},
    {"id": 5, "nome": "Webcam", "prezzo": 89, "categoria": "accessori"},
]
```

```
# Task:
# 1. Crea GET "/prodotti" con query parameter opzionale
#     "categoria" per filtrare i prodotti
# 2. Aggiungi query parameter "prezzo_max" (float, opzionale)
#     per filtrare prodotti con prezzo ≤ prezzo_max
# 3. Aggiungi paginazione con "skip" (default 0) e
#     "limit" (default 10)
```

5. Pydantic

Validazione Dati Automatica

Il Problema dei Dati Sporchi

Nelle API, gli utenti inviano dati (es. JSON via POST). **Mai fidarsi dell'input!**

Cosa può andare storto?

- Campo obbligatorio mancante
- Tipo sbagliato (stringa invece di numero)
- Valore fuori range (età negativa, prezzo = -100)
- Formato errato (email senza @, data invalida)
- **Attacchi:** SQL injection, XSS, dati malevoli

Senza validazione (PERICOLOSO!):

Principio di sicurezza: *"Never trust user input"*

Ogni dato dall'esterno deve essere validato, sanitizzato e tipizzato.

Pydantic: Validazione Dichiarativa

Pydantic definisce la "forma" dei dati usando classi Python con Type Hints.

```
from pydantic import BaseModel
from typing import Optional
class Item(BaseModel):
    name: str                # Obbligatorio, deve essere stringa
    price: float              # Obbligatorio, deve essere numero
    description: Optional[str] = None # Opzionale
    in_stock: bool = True     # Default: True
```

Pydantic: Validazione Dichiarativa

Cosa fa Pydantic automaticamente:

Funzione	Esempio
Valida i tipi	<code>price="ciao"</code> → Errore
Converte dove possibile	<code>price="10.5"</code> → <code>10.5</code> (float)
Applica default	Campo mancante → usa default
Errori dettagliati	Dice QUALE campo e PERCHÉ è invalido
Genera JSON Schema	Per documentazione automatica

Vantaggi rispetto alla validazione manuale:

- Codice dichiarativo (definisci COSA, non COME)
- Errori consistenti e leggibili
- Documentazione sempre aggiornata

Usare i Modelli nelle API

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float
    is_offer: Optional[bool] = None

@app.post("/items/")
def create_item(item: Item):      # ← FastAPI valida automaticamente
    # 'item' è un oggetto Pydantic, non un dict
    return {
        "item_name": item.name,
        "price_with_tax": item.price * 1.22
    }
```

Se invii JSON invalido: FastAPI risponde con errore 422 e dettagli!

Validazione Avanzata con Field

```
from pydantic import BaseModel, Field, EmailStr
from typing import Optional

class User(BaseModel):
    name: str = Field(
        min_length=2,
        max_length=50,
        description="Nome dell'utente"
    )
    email: EmailStr # Valida automaticamente il formato email!
    age: int = Field(ge=0, le=120, description="Età in anni")
    bio: Optional[str] = Field(None, max_length=500)

# Esempio per la documentazione
model_config = {
    "json_schema_extra": {
        "examples": [{
            "name": "Mario Rossi",
            "email": "mario@example.com",
            "age": 30
        }]
    }
}
```


Validatori Field Comuni

Parametro	Tipo	Descrizione
<code>min_length</code>	str	Lunghezza minima stringa
<code>max_length</code>	str	Lunghezza massima stringa
<code>ge</code>	numeri	Greater or Equal ($\geq$)
<code>gt</code>	numeri	Greater Than ($>$)
<code>le</code>	numeri	Less or Equal ($\leq$)
<code>lt</code>	numeri	Less Than ($<$)
<code>pattern</code>	str	Regex pattern
<code>default</code>	tutti	Valore di default

```
from pydantic import Field
```

```
price: float = Field(gt=0, description="Prezzo deve essere positivo")
```

```
sku: str = Field(pattern=r"^[A-Z]{3}-\d{4}$") # Es: "ABC-1234"
```

Modelli Annidati

```
from pydantic import BaseModel
from typing import List, Optional

class Address(BaseModel):
    street: str
    city: str
    zip_code: str

class OrderItem(BaseModel):
    product_id: int
    quantity: int = Field(ge=1)
    price: float

class Order(BaseModel):
    customer_name: str
    shipping_address: Address          # Modello annidato
    items: List[OrderItem]            # Lista di modelli
    notes: Optional[str] = None
```



Esercizi Intermedi: Pydantic

Pratica Prima di Proseguire

Esercizio 4.1: Modelli Pydantic Base

Obiettivo: Definire modelli Pydantic e usarli con endpoint POST.

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import Optional

app = FastAPI()

# Task:
# 1. Crea un modello "Contatto" con:
#     - nome: str (obbligatorio)
#     - email: str (obbligatorio)
#     - telefono: Optional[str] (opzionale, default None)
#     - nota: Optional[str] (opzionale, default None)
# 2. Crea POST "/contatti" che riceve un Contatto e restituisce:
#     {"messaggio": "Contatto creato", "contatto": ...}
# 3. Testa con curl o Swagger UI inviando JSON:
#     {"nome": "Mario", "email": "mario@email.it"}
# 4. Testa cosa succede inviando JSON senza il campo "nome"
```

Esercizio 4.2: Validazione con Field

Obiettivo: Aggiungere vincoli di validazione ai campi del modello.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
from typing import Optional

app = FastAPI()

# Task:
# 1. Crea un modello "Prodotto" con:
#     - nome: str (min 2 caratteri, max 100)
#     - prezzo: float (deve essere > 0)
#     - quantita: int (deve essere ≥ 0)
#     - descrizione: Optional[str] (max 500 caratteri)
# 2. Crea POST "/prodotti" che riceve un Prodotto
#     e restituisce il prodotto con un campo aggiuntivo
#     "totale_valore": prezzo * quantita
# 3. Testa inviando valori non validi:
#     - nome con 1 solo carattere
#     - prezzo negativo
#     - quantita = -5
# Osserva gli errori 422 restituiti da FastAPI
```

Esercizio 4.3: Modelli Annidati

Obiettivo: Costruire modelli complessi con modelli annidati.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
from typing import List
app = FastAPI()
# Task:
# 1. Crea un modello "Indirizzo" con: via (str), città (str),
#    cap (str)
#
# 2. Crea un modello "Riga" con: prodotto (str),
#    quantita (int, ≥ 1), prezzo_unitario (float, > 0)
#
# 3. Crea un modello "Ordine" con:
#    - cliente: str
#    - indirizzo_spedizione: Indirizzo (modello annidato)
#    - righe: List[Riga] (lista di modelli)
#
# 4. Crea POST "/ordini" che riceve un Ordine e restituisce
#    l'ordine con un campo "totale" calcolato dalla somma di
#    (quantita * prezzo_unitario) per ogni riga
```

Modelli Separati: Input vs Output

Best practice: modelli diversi per creazione e risposta.

```
# Modello per CREARE (input dall'utente)
class ItemCreate(BaseModel):
    name: str
    price: float
# Modello per RISPOSTA (include campi generati dal server)
class ItemResponse(BaseModel):
    id: int # Generato dal database
    name: str
    price: float
    created_at: datetime # Generato dal server
@app.post("/items/", response_model=ItemResponse)
def create_item(item: ItemCreate):
    # Simula salvataggio nel DB
    new_item = {
        "id": 1,
        "name": item.name,
        "price": item.price,
        "created_at": datetime.now()
    }
    return new_item
```

6. CRUD Completo

Create, Read, Update, Delete

Pattern CRUD

Le 4 operazioni fondamentali su ogni risorsa. Il **CRUD** è la base di quasi tutte le applicazioni data-driven.

Operazione	HTTP	Endpoint	Descrizione	Status Success
C reate	POST	<code>/items/</code>	Crea nuovo item	201
R ead	GET	<code>/items/</code> o <code>/items/{id}</code>	Leggi tutti o uno	200
U pdate	PUT/PATCH	<code>/items/{id}</code>	Modifica item	200
D elete	DELETE	<code>/items/{id}</code>	Elimina item	204

REST API Design

Design REST best practices:

- Usa **nomi plurali** per le risorse (`/users` , non `/user`)
- L'**ID** va nel path per operazioni su singola risorsa
- **POST** su collection (`/items/`), non su singolo item
- Restituisci l'oggetto creato/modificato nella risposta
- Usa i **status code** corretti (non sempre 200!)

CRUD Completo: Esempio Items

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional, List
```

```
app = FastAPI()
```

```
# "Database" in memoria
```

```
db_items = {}
```

```
counter = 0
```

```
class ItemCreate(BaseModel):
```

```
    name: str
```

```
    price: float
```

```
class ItemUpdate(BaseModel):
```

```
    name: Optional[str] = None
```

```
    price: Optional[float] = None
```

```
class Item(BaseModel):
```

```
    id: int
```

```
    name: str
```

```
    price: float
```

CRUD: Create & Read

```
# CREATE - POST /items/
@app.post("/items/", response_model=Item, status_code=201)
def create_item(item: ItemCreate):
    global counter
    counter += 1
    new_item = Item(id=counter, name=item.name, price=item.price)
    db_items[counter] = new_item
    return new_item

# READ ALL - GET /items/
@app.get("/items/", response_model=List[Item])
def list_items(skip: int = 0, limit: int = 10):
    items = list(db_items.values())
    return items[skip : skip + limit]

# READ ONE - GET /items/{id}
@app.get("/items/{item_id}", response_model=Item)
def get_item(item_id: int):
    if item_id not in db_items:
        raise HTTPException(status_code=404, detail="Item not found")
    return db_items[item_id]
```

CRUD: Update & Delete

```
# UPDATE - PUT /items/{id}
@app.put("/items/{item_id}", response_model=Item)
def update_item(item_id: int, item_update: ItemUpdate):
    if item_id not in db_items:
        raise HTTPException(status_code=404, detail="Item not found")

    stored_item = db_items[item_id]
    update_data = item_update.model_dump(exclude_unset=True)

    for key, value in update_data.items():
        setattr(stored_item, key, value)

    return stored_item

# DELETE - DELETE /items/{id}
@app.delete("/items/{item_id}", status_code=204)
def delete_item(item_id: int):
    if item_id not in db_items:
        raise HTTPException(status_code=404, detail="Item not found")
    del db_items[item_id]
    return None # 204 No Content
```



Esercizi Intermedi: CRUD

Pratica Prima di Proseguire

Esercizio 5.1: CRUD Rubrica Contatti

Obiettivo: Implementare un CRUD completo per una rubrica contatti.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from typing import Optional, List

app = FastAPI()

# Task:
# 1. Crea modello ContattoCreate (nome: str, telefono: str,
#    email: Optional[str])
# 2. Crea modello Contatto (aggiunge id: int)
# 3. Crea un dizionario db_contatti come "database"
# 4. Implementa:
#    - POST /contatti      → crea contatto (status 201)
#    - GET  /contatti      → lista tutti i contatti
#    - GET  /contatti/{id} → singolo contatto (404 se non esiste)
#    - DELETE /contatti/{id} → elimina contatto (204)
```

Esercizio 5.2: Aggiungere Update e Ricerca

Obiettivo: Estendere il CRUD con update parziale e ricerca.

```
# Continua dall'esercizio 5.1

# Task:
# 1. Crea modello ContattoUpdate con tutti i campi opzionali
#    (nome, telefono, email)
#
# 2. Implementa PUT /contatti/{id} che aggiorna solo
#    i campi forniti (usa model_dump(exclude_unset=True))
#
# 3. Aggiungi query parameter "q" a GET /contatti per
#    cercare contatti il cui nome contiene la stringa
#    (case-insensitive)
#
# 4. Aggiungi GET /contatti/count che restituisce
#    {"totale": N} con il numero di contatti
#    ATTENZIONE: metti questo endpoint PRIMA di /{id}!
```


Riepilogo del Modulo

Argomento	Concetti Chiave
API	Interfaccia client-server, REST, risorse
HTTP	GET/POST/PUT/DELETE, status code, headers
FastAPI	Decoratori, routing, uvicorn
Path/Query	Parametri URL dinamici e filtri
Pydantic	Modelli, validazione, Field
CRUD	Create, Read, Update, Delete pattern

Errori Comuni: API Development

Errore	Problema	Soluzione
Dimenticare <code>async</code>	Blocca il server su operazioni I/O	Usa <code>async def</code> per chiamate esterne
Status code sbagliato	<code>200</code> per creazione invece di <code>201</code>	Usa <code>status_code=201</code> per POST
Endpoint <code>/tasks/stats</code> dopo <code>{id}</code>	<code>stats</code> interpretato come ID	Metti endpoint specifici PRIMA
Non validare input	Dati sporchi nel database	Sempre modelli Pydantic

Errori Comuni: API Development

```
# ❌ Ordine sbagliato - stats mai raggiunto!
@app.get("/tasks/{task_id}")
def get_task(task_id: int): ...

@app.get("/tasks/stats") # "stats" matchato come task_id!
def get_stats(): ...

# ✅ Corretto - endpoint specifici prima
@app.get("/tasks/stats") # Prima!
def get_stats(): ...

@app.get("/tasks/{task_id}")
def get_task(task_id: int): ...
```

Laboratorio 7

Progetto Finale: "Meteo Data Hub & API"



Obiettivo del Progetto

Costruire un'applicazione backend **completa** in Python che:

1. **Estrae** dati meteorologici da un servizio esterno
2. **Salva** i dati grezzi in locale (File I/O)
3. **Pulisce e analizza** i dati con Pandas
4. **Espone** i risultati tramite una API REST personalizzata

Questo progetto mette alla prova tutto ciò che hai imparato:

Modulo	Concetti
1-4	Tipi, strutture dati, funzioni, classi OOP
5	Virtual env, pip, File I/O, requests
6	Pandas: pulizia, groupby, export CSV
7	FastAPI, Pydantic, CRUD, path/query params



Struttura del Progetto

```
esercizio_conclusivo/  
├─ main.py           # 🎯 Entry point - App FastAPI con gli endpoint  
├─ fetcher.py        # 📡 Classe OOP per scaricare dati meteo  
├─ analyzer.py       # 📊 Funzione Pandas per analisi e pulizia  
├─ requirements.txt   # 📦 Dipendenze del progetto  
├─ dati_grezzi.json   # 📁 (generato) Dati scaricati dall'API  
└─ report_meteo.csv   # 📁 (generato) Report aggregato per giorno
```

Dipendenze (requirements.txt):

```
requests  
pandas  
fastapi  
uvicorn  
pydantic
```



Step 1: Setup dell'Ambiente

Riferimento: Modulo 5 - Virtual Environment & PIP

Cosa fare:

1. Crea una nuova cartella per il progetto
2. Inizializza un **Virtual Environment** (`.venv`)
3. Installa le librerie necessarie con `pip`
4. Salva le dipendenze in `requirements.txt`



Step 1: Setup dell'Ambiente

Crea la cartella del progetto

```
mkdir esercizio_conclusivo
```

```
cd esercizio_conclusivo
```

Crea e attiva il virtual environment

```
python -m venv .venv
```

```
.venv\Scripts\activate          # Windows
```

Installa le dipendenze

```
pip install requests pandas fastapi "uvicorn[standard]" pydantic
```

Salva le dipendenze

```
pip freeze > requirements.txt
```




Step 2: Acquisizione Dati

Riferimento: Moduli 1, 2, 3, 4, 5 - OOP, requests, File I/O

Crea il file `fetcher.py` con una classe `MeteoClient` :

1. **Costruttore (`__init__`)**: Definisci l'URL base dell'API pubblica [Open-Meteo](#)
2. **Metodo** `ottieni_previsioni(lat, lon)` :
 - Usa `requests.get()` per scaricare dati orari (temperatura)
 - Passa i parametri come **Query Parameters**
 - Gestisci errori con `try...except` (connessione, status code)
 - Salva il JSON di risposta in `dati_grezzi.json` (File I/O)



Step 2: Soluzione

fetcher.py

```
import requests
import json

class MeteoClient:
    """Classe per scaricare i dati meteo da Open-Meteo."""
    def __init__(self):
        self.base_url = "https://api.open-meteo.com/v1/forecast"

    def ottieni_previsioni(self, lat: float, lon: float) → dict:
        """Scarica i dati meteo e li salva in un file JSON."""
        parametri = {
            "latitude": lat, "longitude": lon,
            "hourly": "temperature_2m", "timezone": "auto"
        }
        try:
            risposta = requests.get(
                self.base_url, params=parametri, timeout=10
            )
            risposta.raise_for_status()
            dati = risposta.json()

            with open("dati_grezzi.json", "w", encoding="utf-8") as file:
                json.dump(dati, file, indent=4)

            print("✅ Dati scaricati in 'dati_grezzi.json'")
            return dati
        except requests.exceptions.RequestException as e:
            print(f"❌ Errore HTTP: {e}")
            raise
```



Step 3: Analisi e Pulizia Dati

Riferimento: Modulo 6 - Pandas, GroupBy, Export

Crea il file `analyzer.py` con una funzione `analizza_e_salva_dati()` :

1. **Leggi** il file `dati_grezzi.json` e convertilo in DataFrame
2. **Pulizia**: Controlla valori nulli (`NaN`) nelle temperature e riempi con la media (`fillna`)
3. **Manipolazione e GroupBy**: Raggruppa i dati orari per **giorno** e calcola:
 - Temperatura **media** giornaliera
 - Temperatura **massima** e **minima** del giorno
4. **Export**: Salva il DataFrame riassuntivo in `report_meteo.csv`



Step 3: Soluzione

analyzer.py

```
import pandas as pd
import json

def analizza_e_salva_dati():
    """Legge il JSON, pulisce, raggruppa per giorno e salva CSV."""
    with open("dati_grezzi.json", "r", encoding="utf-8") as file:
        dati_json = json.load(file)

    orari = dati_json["hourly"]["time"]
    temp_celsius = dati_json["hourly"]["temperature_2m"]

    # BONUS: List Comprehension Celsius → Fahrenheit
    temp_f = [(t * 9/5) + 32 if t is not None else None
              for t in temp_celsius]

    df = pd.DataFrame({"data_ora": orari, "temperatura_F": temp_f})

    # Pulizia: fillna con la media
    df["temperatura_F"] = df["temperatura_F"].fillna(
        df["temperatura_F"].mean()
    )

    # Estrai il giorno dalla data_ora
    df['giorno'] = df['data_ora'].str.split('T').str[0]

    # GroupBy per giorno
    df_riassunto = df.groupby('giorno').agg(
        temp_media=('temperatura_F', 'mean'),
        temp_max=('temperatura_F', 'max'),
        temp_min=('temperatura_F', 'min')
    ).reset_index().round(2)

    df_riassunto.to_csv("report_meteo.csv", index=False)
    print("✅ Report salvato in 'report_meteo.csv'")
```



Step 4: Costruzione dell'API REST

Riferimento: Modulo 7 - FastAPI, Pydantic, CRUD

Crea il file `main.py` con un'app FastAPI e 3 endpoint:

Metodo	Endpoint	Descrizione
POST	<code>/aggiorna-dati</code>	Riceve coordinate (Pydantic), scarica dati e genera report
GET	<code>/report-completo</code>	Legge <code>report_meteo.csv</code> e lo restituisce come JSON
GET	<code>/meteo/{giorno}</code>	Path parameter - statistiche di un giorno specifico



Step 4: Costruzione dell'API REST

Elementi chiave da implementare:

- **Pydantic Model** `Coordinate` per validare `latitudine` e `longitudine`
- **HTTPException 404** se il report non esiste o il giorno non è trovato
- **HTTPException 500** per errori interni (try/except)
- **Import** di `fetcher.py` e `analyzer.py` come moduli



Step 4: Soluzione `main.py` (Parte 1)

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import pandas as pd
import os
from fetcher import MeteoClient
from analyzer import analizza_e_salva_dati

app = FastAPI(title="Meteo Data Hub & API")
# Modello Pydantic per validare il Body della POST
class Coordinate(BaseModel):
    latitudine: float
    longitudine: float
@app.get("/")
def root():
    """Endpoint di benvenuto."""
    return {
        "messaggio": "Benvenuto nel Meteo Data Hub!",
        "istruzioni": "Vai su /docs per esplorare le API."
    }
```



Step 4: Soluzione `main.py` (Parte 2)

```
@app.post("/aggiorna-dati")
def aggiorna_dati(coords: Coordinate):
    """Scarica nuovi dati meteo e genera il report."""
    client = MeteoClient()
    try:
        client.ottieni_previsioni(coords.latitudine, coords.longitudine)
        analizza_e_salva_dati()
        return {"messaggio": "Successo! Dati aggiornati e report generato."}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Errore interno: {str(e)}") from e

@app.get("/report-completo")
def get_report_completo():
    """Ritorna il report CSV completo in formato JSON."""
    if not os.path.exists("report_meteo.csv"):
        raise HTTPException(status_code=404, detail="Report inesistente. Chiama /aggiorna-dati prima.")
    df = pd.read_csv("report_meteo.csv")
    return df.to_dict(orient="records")
```




Step 4: Soluzione `main.py` (Parte 3)

```
@app.get("/meteo/{giorno}")
def get_meteo_giorno(giorno: str):
    """
    Statistiche meteo per un giorno specifico.
    Formato atteso: YYYY-MM-DD (es. 2024-01-15)
    """
    if not os.path.exists("report_meteo.csv"):
        raise HTTPException(status_code=404, detail="Report inesistente.")

    df = pd.read_csv("report_meteo.csv")
    df_filtrato = df[df['giorno'] == giorno]

    if df_filtrato.empty:
        raise HTTPException(
            status_code=404,
            detail=f"Dati non trovati per il giorno: {giorno}"
        )
    return df_filtrato.to_dict(orient="records")[0]
```



Come Testare il Progetto

Apri Swagger UI: `http://127.0.0.1:8000/docs`

Oppure con curl:

```
# 1. Aggiorna dati (Roma: 41.89, 12.48)
curl -X POST http://127.0.0.1:8000/aggiorna-dati \
  -H "Content-Type: application/json" \
  -d '{"latitudine": 41.89, "longitudine": 12.48}'
```

```
# 2. Vedi report completo
curl http://127.0.0.1:8000/report-completo
```

```
# 3. Dettaglio di un giorno specifico
curl http://127.0.0.1:8000/meteo/2024-01-15
```

```
# 4. Giorno inesistente → 404
curl http://127.0.0.1:8000/meteo/2099-12-31
```



Bonus (Opzionale)

Per chi finisce prima:

- **List Comprehension:** Prima di passare i dati a Pandas, converti le temperature da **Celsius a Fahrenheit** $(C \times 9/5) + 32$

```
temperature_f = [  
    (temp * 9/5) + 32 if temp is not None else None  
    for temp in temperature_celsius  
]
```

- **Moduli e Namespace:** Assicurati che il codice sia ben diviso in moduli (`fetcher.py` , `analyzer.py`) e importato correttamente in `main.py`

```
from fetcher import MeteoClient      # Classe dal modulo fetcher  
from analyzer import analizza_e_salva_dati # Funzione dal modulo analyzer
```



Corso Completato!

Hai completato il **Corso Python!**

Competenze acquisite:

- ☒ Data Science con NumPy e Pandas
- ☒ API REST con FastAPI
- ☒ Validazione con Pydantic

Buon coding! 🚀